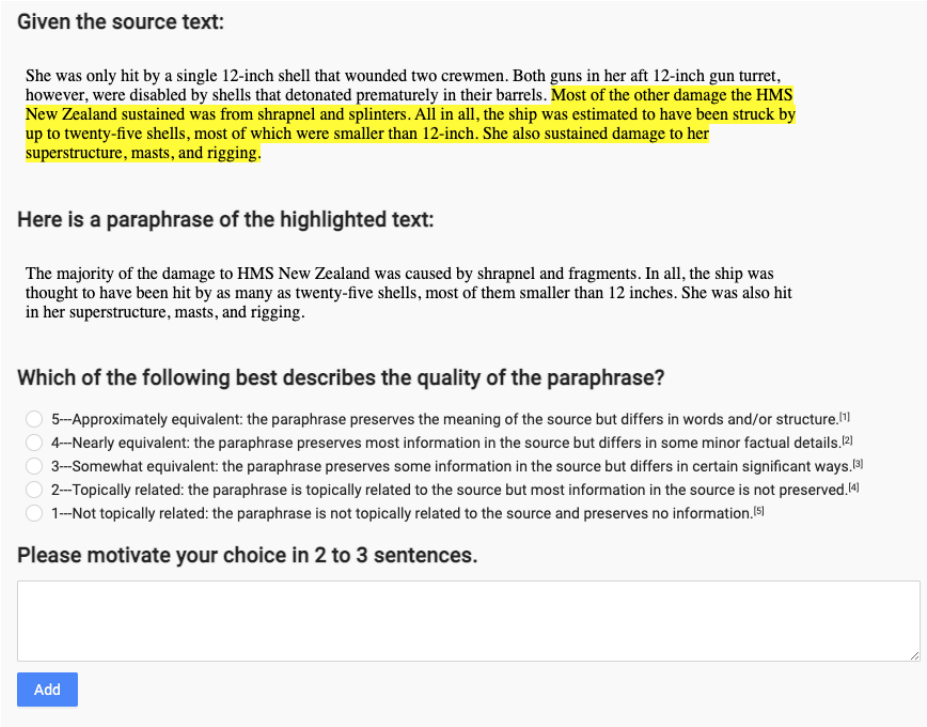

Figure 7: The interface of the annotation platform used in our human study

Table 4: This table shows how often each point in the Likert scale was chosen by 3 annotators for the pairs of original and paraphrased texts. Twenty text pairs are randomly selected for each lexical code (L). 81.8% of the time, our model DIPPER provides a paraphrase which is nearly equivalent to the input in terms of semantic meaning.

L	Sum of 4 and 5	5 Approx. equivalent	4 Nearly equivalent	3 Somewhat equivalent	2 Topically related
20	95.0%	63.3%	31.7%	5.0%	0.0%
40	78.3%	45.0%	33.3%	21.7%	0.0%
60	70.0%	28.3%	41.7%	28.3%	1.7%
Total	81.1%	45.6%	35.6%	18.3%	0.6%

Strengths First, the third example in Table 8 exemplifies DIPPER's ability to leverage information from context to increase diversity while maintaining coherence (i.e., from *line... reference the song's title* to *reference to "I'm the Greatest"*). The same is observed in row 2 where DIPPER uses the context to interchange *he* and *Churchill*. A paraphrase model without looking into context will have great difficulty in doing this and no prior paraphraser (see Table 5 for a list) is capable of that. Second, the example in the fifth row highlights DIPPER's ability to make significant changes to original texts with a high lexical diversity code ($L60$) (see the color coding) while preserving their semantic meaning as rated by the annotators.

Qualitative shortcomings: The first shortcoming is that, when the original text contains new created proper names (unlike common people and country names), such as the ones in row 6 (*Homing Attack* and *Slide Attack*), a high lexical code has a tendency to change such nouns, leading to the result that one of our annotators deems it to be only topically related to the original. However, this shortcoming can be overcome by decreasing the lexical code, which a user can choose from a continuous range (from 0 to 100). For instance, in row 1 with `lex=20`, the songs' names *M's Confession* and *Gone Fishing* are kept intact. Another shortcoming is that DIPPER occasionally omits content from an original text. While in some cases such removal is acceptable (see row 6), in other cases it causes significant change in the meaning of the text (see row 4). However, the former case can be overcome by paraphrasing a shorter paragraph at a time.

Overall, the human study shows that DIPPER performs well at preserving the semantic meaning of original texts while introducing both semantic and syntactic diversity. Because DIPPER provides user-friendly controllabilty of output diversity, a user can adjust the control code to find the most suitable paraphrase for their need.

D Related work for discourse paraphrasing

D.1 Survey of paraphrase generation papers

As an important NLP task, paraphrasing has attracted much attention. Many models have been proposed to improve the quality of paraphrases. To position our model DIPPER and highlight its strengths, we conduct a survey of paraphrase generation papers from 2018 to 2022 (Table 5) and focus on the following four aspects:

1. Whether a model can paraphrase a paragraph at once,
2. whether a model can merge or split consecutive sentences when appropriate,
3. whether a model leverages context surrounding an input sentence when paraphrasing,
4. whether a model provides control knobs for users to customize the output diversity.

The survey shows that only three out of 25 papers mentioned that their model can paraphrase more than one sentence (but not necessarily at once). None of them enables their model to merge or split sentences when paraphrasing. No model uses information from context surrounding an input sentence during inference time. Finally, 14 papers offer ways for users to customize the diversity of paraphrases. However, most diversity control methods such as constituency parses or exemplars may not be straightforward and intuitive to end-users as the scalar control knobs in DIPPER.

In contrast to the papers in the survey, DIPPER nicely combines all desiderata into one model and offers intuitive control knobs for lexical and syntactic diversity. Automatic and human evaluation show that DIPPER can efficiently leverage context information and reorganize sentences while having high fidelity in meaning (Appendix C).

D.2 Other related work

In this section we discuss a few additional less related papers which were not included in our survey in Appendix D.1. Our discourse paraphraser is closely related to work on contextual machine translation, where source/target context is used to improve sentence-level machine translation [House, 2006, Jean et al., 2017, Wang et al., 2017, Tiedemann and Scherrer, 2017, Kuang et al., 2018, Agrawal et al., 2018, Miculicich et al., 2018, Zhang et al., 2018, Xiong et al., 2019, Jean et al., 2019, Voita et al., 2019a, Yin et al., 2021, Mansimov et al., 2021]. Prior work has shown that context helps with anaphora resolution [Voita et al., 2018], deixis, ellipsis, and lexical cohesion [Voita et al., 2019b]. Efforts to make paraphrase generation more contextual have been quite limited. A few efforts have attempted to use sentence level context to paraphrase phrases [Connor and Roth, 2007, Max, 2009], and dialogue context to paraphrase individual dialogues in a chat [Garg et al., 2021].

Our work is also related to efforts in text simplification to go beyond a sentence, by collecting relevant datasets [Xu et al., 2015, Devaraj et al., 2021] and building unsupervised algorithms [Laban et al., 2021]. Note that our work focuses on a general-purpose paraphrasing algorithm and is not tied to any particular style, but could be utilized for document-level style transfer using techniques like Krishna et al. [2020, 2022b]. Similar efforts have also been undertaken in machine translation, [Popescu-Belis et al., 2019, Junczys-Dowmunt, 2019, Maruf et al., 2021], attempting to translate paragraphs/documents at once.

E More background on detectors of AI-generated text

In this section, we provide an overview of existing algorithms that have been developed for the purpose of detecting machine-generated text. Such algorithms fall into three main categories: (1) watermarking algorithms, which modify the generative algorithm to encode hidden information unique to the API (Appendix E.1); (2) statistical outlier detection methods, which do not modify

Table 5: The table shows the result of our survey of paraphrase generation papers from 2018 to 2022. We focus on four aspects: (1) whether a model can paraphrase multiple sentences at once, (2) whether a model is able to merge or split an input sentence when appropriate, (3) whether a model takes context surrounding the input sentence into consideration when paraphrasing, and (4) whether a model enables users to control the semantic and syntactic diversity of paraphrases. [1]Granularity levels are *word*, *phrase*, and *sentence*. [2]Meng et al. [2021] use context for their dataset construction, but do not leverage it during training/inference. [3]The diversity score is a combination of the unigram Jaccard distance and the relative position change for unigrams. [4]The code is represented by a three dimensional vector corresponding to semantic similarity as well as syntactic and lexical distances between the input and output sentences.

Paper	Multi-sentence	Merge / Splits	Contextual	Diversity Control
Iyyer et al. [2018]	✗	✗	✗	Constituency parse
Li et al. [2018]	✗	✗	✗	✗
Roy and Grangier [2019]	✗	✗	✗	✗
Witteveen and Andrews [2019]	✓	?	✗	✗
Kumar et al. [2019]	✗	✗	✗	✗
Hu et al. [2019]	✗	✗	✗	Decoding constraints
Chen et al. [2019]	✗	✗	✗	Exemplar
Li et al. [2019]	✗	✗	✗	Granularity control[1]
Goyal and Durrett [2020]	✗	✗	✗	Exemplar
Lewis et al. [2020]	✓	?	✗	✗
Thompson and Post [2020]	✗	✗	✗	n-gram overlap
Kumar et al. [2020]	✗	✗	✗	Exemplar
Kazemnejad et al. [2020]	✗	?	✗	✗
Krishna et al. [2020]	✗	✗	✗	✗
Rajauria [2020]	✗	✗	✗	✗
Meng et al. [2021]	✗	✗	✗[2]	Diversity score[3]
Huang and Chang [2021]	✗	✗	✗	Constituency parse
Lin et al. [2021]	✓	✗	✗	✗
Goutham [2021]	✗	✗	✗	✗
Damodaran [2021]	✗	✗	✗	Binary
Dopierre et al. [2021]	✗	✗	✗	n-gram
Bandel et al. [2022]	✗	✗	✗	Control code[4]
Hosking et al. [2022]	✗	✗	✗	Syntactic sketch
Yang et al. [2022]	✗	✗	✗	Examplar+Keywords
Xie et al. [2022]	✗	✗	✗	✗
DIPPER (ours)	✓	✓	✓	✓

the generative algorithm but look for inherent artifacts in generated text (Appendix E.2); and (3) classifiers trained to discriminate machine-generated text from human-written text (Appendix E.3). Finally, in Appendix E.4, we compare and contrast our work to Sadasivan et al. [2023], who also note the efficacy of paraphrasing attacks but do not consider a retrieval-based defense in their pessimistic conclusion about the fate of AI-generated text detection.

E.1 Watermarking language model outputs

A "watermark" is a modification to the generated text that can be detected by a statistical algorithm while remaining imperceptible to human readers. Effective watermarks are difficult to remove and have little effect on the quality of generated text. Prior work attempted to watermark natural language using syntax tree manipulations [Topkara et al., 2005, Meral et al., 2009], and this area has gotten renewed interest with large language models generating human-like text [Abdelnabi and Fritz, 2021, Grinbaum and Adomaitis, 2022]. Most recently, Kirchenbauer et al. [2023a] propose a simple algorithm that only requires access to the LLM's logits at each time step to add watermarks. The watermark can then be verified with only blackbox access to the LM and knowledge of a specific hash function. This algorithm operates in three steps: